PANINI CLI

Technical overview and live demo guide

A cost-aware, contained, evidence-first terminal coding agent

```
$ panini-cli route "what does calc.py do?"
  ollama/qwen2.5:3b-instruct   local · $0
$ panini-cli exec "fix the failing check" --check "python check_text.py"
  cloud tier · check passed in MXC · about $0.06
```

Webinar companion document · PANINI CLI 2.1.1b1 (Windows public beta) · October 2026
github.com/suryacharanteja/panini-cli · MIT License

Contents

1. At a glance . . . 3
2. Capabilities . . . 4
3. How PANINI differs from Codex CLI and Claude Code . . . 5
Where PANINI is genuinely different . . . 5
Where the others are ahead today . . . 5
4. Architecture . . . 6
5. Concept: routing with Jev . . . 8
Selection . . . 8
What is Jev? . . . 8
Escalation and completion checks . . . 9
6. Concept: contained execution on Windows . . . 9
Policy identity and approvals . . . 9
Network exceptions . . . 10
7. Concept: approvals, trust and secrets . . . 10
8. Concept: evidence — verification, checkpoints, receipts . . . 10
9. Concept: cost, context and extensibility . . . 11
Cost ledger and budgets . . . 11
Context management . . . 11
Extensibility . . . 11
10. Evidence: what has been measured . . . 12
10.1 Routing eval (10 small tasks, test oracles) . . . 12
10.2 Harbor pilot (6 tasks × 3 repeats, Linux containers) . . . 12
10.3 Terminal-Bench readiness . . . 12
10.4 Published reference results for other agents (context only) . . . 12
10.5 Engineering verification . . . 13
11. Live demo script (about 25 minutes) . . . 14
Before the webinar (do this 30 minutes ahead) . . . 14
Rehearsal results (2026-10-11, development machine) . . . 17
If something goes wrong . . . 17
12. Limitations and roadmap . . . 18
Limitations to state openly . . . 18
Roadmap . . . 18
13. Q&A preparation . . . 18
14. Appendix: commands and configuration . . . 20

1 At a glance

PANINI CLI is a terminal coding agent for Windows. Like Codex CLI and Claude Code, it reads a project, plans, edits files, runs commands and reports back. It is built around three ideas that shape every design decision:

Idea	What it means	How it shows up
1. Spend tokens only when the task needs them	Each user turn is routed to the cheapest model that can finish it, across providers: a local Qwen 2.5 on Ollama ($0, private) or OpenAI GPT-5.6 Luna, Terra or Sol.	Jev capability scoring, an escalation cascade, a per-turn cost ledger and session budgets.
2. Contain execution by default	Commands run inside Microsoft MXC (a Windows BaseContainer) with workspace-only file access, protected secrets, no network unless an exact host is granted, and process limits.	Execution broker, frozen per-task policy, approvals bound to that policy, exact-host CONNECT proxy.
3. Prove, don't claim	"Done" requires evidence: a project check that passed, a grounded answer, a receipt for every action.	Verification gate (`--check`), git checkpoints, hash-chained receipts, public eval traces.

Audience and how to use this document

- **Sections 2–4** are the talk: what PANINI is, how it compares with Codex CLI and Claude Code, and the architecture.
- **Sections 5–9** explain the technical concepts in depth, for the Q&A and for technical attendees.
- **Section 10** is the evidence: what has been measured, and what has not.
- **Section 11** is a timed, copy-paste demo script with a fallback plan. **Section 12** lists honest limitations; **Section 13** prepares likely questions.

> **One-sentence pitch**
> PANINI CLI is a coding agent that routes every task to the cheapest model able to finish it — local or cloud — runs what it does inside a Windows container, and only calls work done when it has evidence.

2 Capabilities

Capability	Summary
Per-task routing	Local Qwen 2.5 → GPT-5.6 Luna → Terra → Sol. Jev rates each candidate; the cheapest above its threshold wins. Explicit choices (`-m`, `/model`, "use qwen") always win.
Escalation cascade	Bad local turns retry on a stronger local model, then the cloud, keeping applied changes and tool results.
Local scouts	Cloud models can delegate read-only exploration (`explore`) to the free local model and receive path:line notes.
Background workers	Read-only explorers, or editors working in a detached git worktree; results and diffs are reviewed, never auto-merged.
Windows containment	Microsoft MXC BaseContainer: workspace grants, secret exclusions, minimal environment, proxy-only network, Job-object limits.
Approvals	Modes read-only · ask · edits · auto. Grants bind command, task, folder, scope and policy identity.
Verification gate	Detects pytest, ruff, npm test, cargo test, go test, dotnet test; `--check` adds acceptance commands. Unmet checks end as *unverified* with a non-zero exit.
Git checkpoints	Snapshots per turn in `refs/panini/`; `/undo` reverts command side effects too; `/diff`, `/review`, `/commit`.
Cost ledger	$ per turn and session, budget stop, `panini-cli usage` with spend by model and local work.
Receipts	Hash-chained record of every tool call; `receipts ID --export report.md`.
Context	Auto-compaction at 80% of the window (summary by the local model), `/compact`, nested AGENTS.md.
Extensibility	MCP client (stdio and HTTPS, OAuth) and MCP server; skills; hooks; JSON-RPC app-server with threads.
Web, images, previews	Approval-gated web fetch, opt-in hosted web search, image input, authenticated previews of built web apps.
Automation	`exec --json`, `--output-schema`, stdin prompts, exit codes 0 / 1 / 3 / 4 / 130.
Distribution	Single unsigned `panini-cli.exe` (Python runtime and sandbox engine embedded) or a Python 3.12 wheel.

3 How PANINI differs from Codex CLI and Claude Code

All three are terminal agents that read code, edit files and run commands under user approval. The differences are in **who chooses the model**, **how execution is contained on Windows**, and **how results are evidenced**.

> **Before presenting this comparison**
> The Codex CLI and Claude Code columns summarize their public documentation as understood in October 2026, at a deliberately coarse level. Both products change quickly. Check their current release notes before the webinar, and present the PANINI column as the part you can demonstrate live.

Dimension	PANINI CLI	Codex CLI	Claude Code
Model choice	**Per turn, across providers.** Router picks local Qwen or a GPT-5.6 tier; escalates on failure.	User selects an OpenAI model per session or profile; other providers via configuration.	Anthropic Claude models; user selects the model, with product-defined defaults.
Local / offline models	**First-class tier.** Works without an API key for local-capable work; $0 local tokens are tracked.	Possible through open-source provider configuration.	Not the primary design focus.
Cost visibility	**$ per turn and session, budgets, usage report.**	Usage reporting through the provider.	Cost and usage reporting in the product.
Windows containment	**Microsoft MXC BaseContainer by default**, exact-host network proxy, capability checks fail closed.	Platform sandboxes (macOS, Linux); Windows support has been evolving.	Permission system; OS-level sandboxing on supported platforms.
Approvals	Modes plus grants bound to task, folder, scope and **policy identity**.	Approval policies and sandbox modes.	Permission modes, allow/deny rules, plan mode.
Evidence	**Verification gate, hash-chained receipts**, git checkpoints.	Session logs; undo support.	Checkpoints / rewind; transcripts.
Extensibility	MCP client and server, skills, hooks, JSON-RPC app-server.	MCP, AGENTS.md, programmatic execution.	MCP, hooks, skills, subagents, SDK, CLAUDE.md.
Maturity	Public beta (2.1.1b1), one primary test host, no independent security audit, unsigned.	Established product with a large user base and IDE / cloud integrations.	Established product with a large user base and IDE integrations.
Benchmark standing	No official Terminal-Bench score yet; evaluation request pending.	Official Terminal-Bench results published.	Official Terminal-Bench results published.

Where PANINI is genuinely different

- **Routing is the product, not a setting.** Codex CLI and Claude Code optimise for one provider's best model. PANINI treats the model as a per-turn resource decision, and measures whether that decision lost solves.
- **Windows is the primary platform.** MXC containment, Windows Credential Manager for keys, ConPTY terminals and a single .exe are first-class, not ports.
- **Evidence is part of completion.** A turn that changed code but never passed a relevant check ends *unverified*, and receipts make the action history tamper-evident.

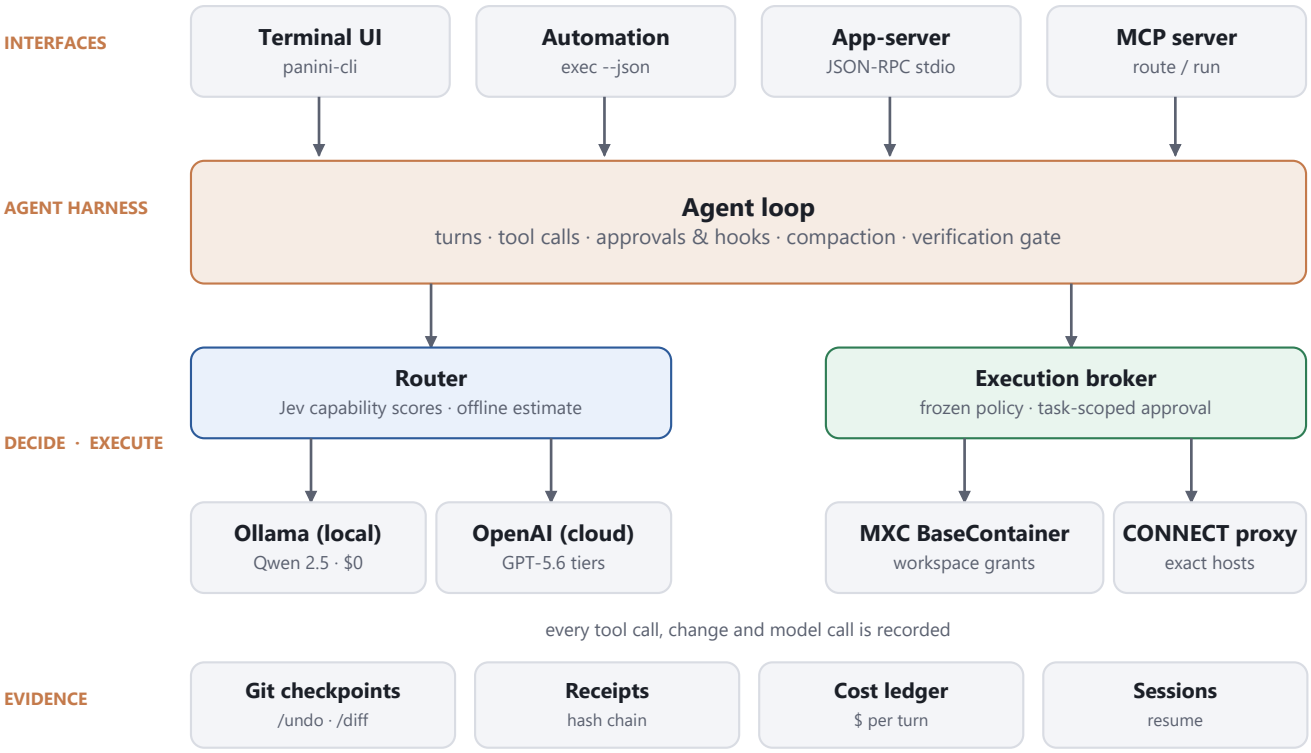
Where the others are ahead today

- **Measured capability on hard tasks.** Both have published official Terminal-Bench results (Section 10.4). PANINI has none yet.
- **Maturity and ecosystem:** IDE integrations, cloud execution, large user bases, signed releases, cross-platform sandboxes.
- **Security assurance:** PANINI's MXC integration has had engineering review and UAT on one machine, not an independent audit.

> **Say this**
> "PANINI is not trying to out-model Codex or Claude Code. It asks a different question: for this specific turn, what is the cheapest model that can finish it — and how do we prove it did?"

4 Architecture

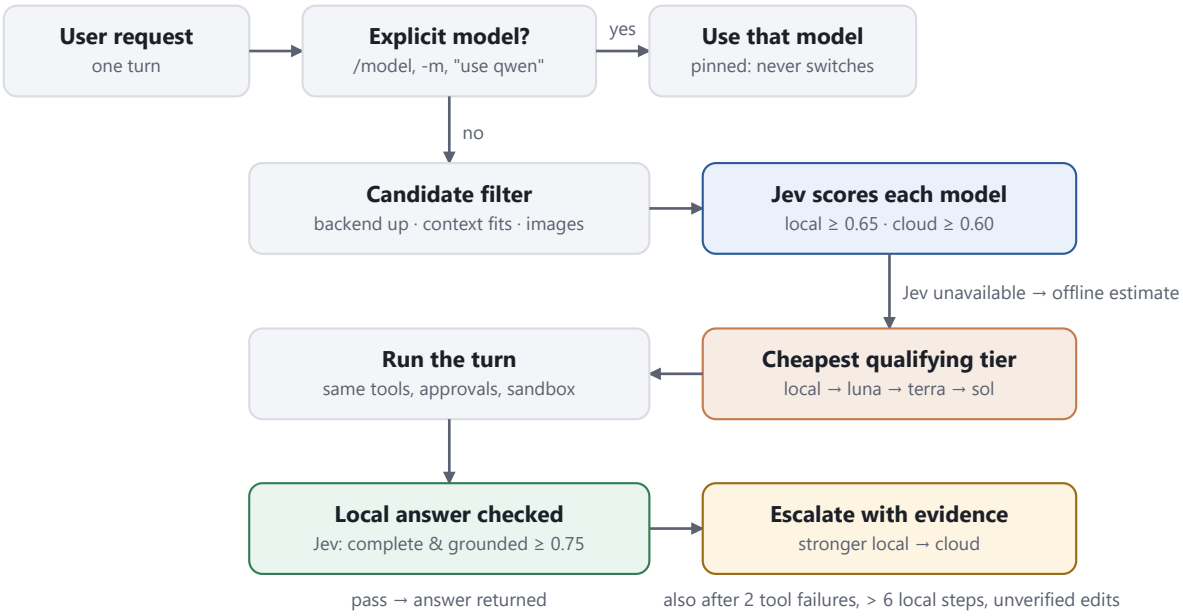
The harness owns tools, approvals, hooks, the sandbox, receipts, state, verification and interruption. Model providers implement one `respond()` interface. Models only construct function calls; the router never constructs commands or task-specific actions. This separation is what makes cross-provider routing safe.



Layer	Key modules	Responsibility
Interfaces	`cli.py, ui.py, server.py, mcp.py`	Interactive terminal, exec, JSON-RPC app-server (threads, turns, steer, interrupt), MCP server.
Agent loop	`loop.py, compaction.py, verify.py, completion.py`	Turns and tool calls, read concurrency with write barriers, compaction, verification and completion checks.
Routing	`model_router.py, complexity.py, jev/`	Candidate discovery, Jev scoring, offline estimate, 60-second decision cache, escalation.
Providers	`model.py, ollama.py`	OpenAI Responses API (stateless, encrypted reasoning carried forward) and Ollama chat, with neutral function-call history.
Execution	`execution.py, mxc_runtime.py, network_proxy.py, terminals.py`	Execution broker, verified MXC engine, exact-host proxy, pipe and ConPTY terminals.
Evidence	`gitops.py, receipts.py, ledger.py, sessions.py, recovery.py`	Turn checkpoints, receipt chain, cost ledger, sessions, opt-in file recovery.

5 Concept: routing with Jev

Routing happens **once per user turn**; every tool step in that turn stays on the chosen model, so a task never bounces between providers mid-flight. The only mid-turn move is an escalation upward.



Selection

- **Explicit choices win.** `-m gpt-5.6-terra`, `/model local` or "use qwen" in the prompt pins the model. Pinned local models never switch providers automatically.
- **Candidates are filtered** by backend availability, image support, session cooldown after backend errors (60 s), and local context capacity: the local window (default 16,384 tokens) minus a 6,000-token reserve for instructions and tools.
- **Jev** independently rates each candidate's ability to finish the request with the shared tools and current context. The cheapest tier above its threshold wins: **local ≥ 0.65, cloud ≥ 0.60**. These are policy thresholds, not calibrated guarantees.
- **Offline estimate.** When Jev is unavailable, a free deterministic workload estimate picks the tier.
- Decisions are cached for 60 seconds, keyed by eligible providers and context.

What is Jev?

Jev is an evaluation model reached through the Vercel AI Gateway (`typesafe-ai/jev`). PANINI asks it independent yes/no questions — "should this exact model handle this request?" — and reads back a probability per model. A warm Node sidecar keeps latency low; requests time out after 3 seconds and fall back to the offline estimate. Jev input is billed separately and is small (fractions of a cent per decision).

Escalation and completion checks

Trigger	Action
Provider error, invalid tool call, malformed arguments, empty reply	Retry on the configured stronger local model, then the cloud fallback.
Two observed tool failures in the turn	Escalate, continuing from recorded results.
More than six local model steps	Escalate, keeping applied changes.
Edits finished without a successful relevant check, after one reminder	Escalate (or end *unverified* if pinned).
Local tool-backed final answer	Jev checks **complete** and **grounded** against the request and tool evidence; both must reach 0.75.
User denied an action	No quality-driven escalation: a denial is the user's decision, not a model failure.

Key idea
Escalation keeps the evidence. Files already changed and tool outputs already recorded stay in the history, so the stronger model continues from facts instead of starting over.

6 Concept: contained execution on Windows

PANINI Sandbox is built from the Microsoft MXC 1.0.0 source and bundled into the executable. On first use the verified engine is installed outside any writable project (`~/.panini/runtime`) and checked against a manifest of SHA-256 digests. `sandbox status` probes the host's real capabilities and refuses to run if a required guarantee is missing; no failure silently retries a command with host access.

Area	Policy inside MXC
Project files	Workspace read/write for coding; read-only for exploration.
Control metadata	`.git` and `.panini` are read-only to arbitrary commands.
Secrets	Recognised secret files (including nested ones) are denied at launch with native filesystem exclusions.
Toolchains	Explicit install roots read-only (Python, Node, Git, …).
Environment	Allowlisted essentials, scratch home and temp; no API tokens, database URLs or credential-agent sockets.
Network	Offline by default. Approved exact public hosts through a CONNECT proxy on port 443; private and loopback addresses, including Ollama and PANINI itself, are denied.
UI	Clipboard and input injection denied.
Process tree	Job object: kill on close, 64 processes, 2 GiB memory, 50% CPU; timeout and 4 MiB output limit.
Terminals	Up to four sessions, 200,000 retained characters, one-hour default lifetime; ConPTY with tty=true.

Policy identity and approvals

The execution broker freezes a policy per task: workspace, read-only mode, toolchain roots, network hosts, resource limits and the engine build. Its **identity** changes whenever any of these change. Approvals are keyed by command, task, folder, execution scope and that identity, so an approval given offline in the workspace cannot be replayed with network access or on the host.

Network exceptions

The agent can request exact hosts with a reason (`request_network_access`), or you grant them up front with `--network-host registry.npmjs.org`. The proxy resolves DNS, rejects private or mixed answers, and connects only to the validated public address. It does not inspect encrypted payloads, so an allowed host can receive project data, and allowed registries can host malicious packages.

> **Be precise in the webinar**
> MXC is a container, not a virtual machine. It does not set disk quotas or protect against kernel vulnerabilities. The proxy currently runs as an unpackaged host process — Microsoft's lowest-assurance proxy deployment. The integration has passed UAT on the development machine but has not had an independent security audit.

7 Concept: approvals, trust and secrets

Mode	File changes	Commands
read-only	Declined	Sandboxed read-only work only
ask (default)	Each change shows a diff and asks	Workspace commands run under the task's MXC policy; host commands and new network hosts ask
edits	Applied without asking	As in ask
auto	Applied	Approved — use only in disposable workspaces

- **Session grants:** answering *a* allows a command for the session — bound to task, folder, scope and policy.
- **Repository trust:** project hooks run only after you trust their configuration, and then inside the task sandbox. Only your own `~/.panini/config.toml` can change approvals or sandboxing — a cloned repository cannot.
- **Git bookkeeping** runs with filters, signing, fsmonitor, hooks and external diff disabled, because a repository's own git configuration can run programs.
- **Secrets:** file tools and `@file` mentions refuse recognised secret files; keys live in Windows Credential Manager (`auth login`), never in a project folder. Arbitrarily named or embedded secrets are not detected.
- **Prompt injection:** file contents, command output, web pages and MCP results are framed as data, never instructions.

8 Concept: evidence — verification, checkpoints, receipts

Verification gate

PANINI detects the project's checks (pytest, ruff, npm test/lint/typecheck, cargo test, go test, dotnet test) and adds them to the agent's instructions. `--check COMMAND` declares explicit acceptance checks. Only a successful relevant check counts — `pwd` or an unrelated command does not. If the agent tries to finish edited work without one, it is reminded once; unmet checks end the turn as **unverified** with a non-zero exit status.

Git checkpoints

Every turn is bracketed by snapshots built from a private index and stored under `refs/panini/`; your index and branches are untouched. `/undo` restores what the last turn changed — including files changed by commands — and skips any file you edited since. `/diff` shows the session, `/review` asks a strong model to review it read-only, `/commit` drafts a message locally and waits for your confirmation.

Receipts

Each tool call writes started / approval / completed events to an append-only log. Every record carries the SHA-256 of the previous one, so editing or deleting a record breaks the chain. Receipts store digests of arguments and outputs, not the raw data. `panini-cli receipts ID --export report.md` verifies the chain and writes a readable report. Limitation: proving that a trailing record was not removed needs an externally trusted head.

9 Concept: cost, context and extensibility

Cost ledger and budgets

Every model call is priced from a table of list prices (overridable under `[prices]`) and appended to `usage.jsonl` in PANINI's data folder. The CLI shows $ per turn and per session; `budget_usd` stops a session at a priced spend; `panini-cli usage` reports spend by model, tokens handled locally and escalations. These are estimates — provider billing is authoritative.

Model (tier)	Input $/M tokens	Output $/M tokens
ollama/qwen2.5:3b-instruct (local)	0	0 (your hardware)
gpt-5.6-luna (cheap)	0.20	1.20
gpt-5.6-terra (balanced)	2.00	12.00
gpt-5.6-sol (strong)	5.00	30.00

GPT-5.6 prices are those recorded in PANINI's price table (sourced from third-party pricing guides, October 2026).

Context management

- Auto-compaction at 80% of the context budget replaces older history with a structured summary (goal, done, decisions, state, next) — written by the local model when available, so it is free. `/compact` does it on demand.
- Instructions come from `~/.panini/AGENTS.md` and every `AGENTS.md` from the repository root to the workspace; deeper ones are listed for the agent to read before working there.
- **Local scouts:** cloud models can call `explore`; a read-only local sub-agent returns path:line notes so cloud tokens go to the edit, not to reading files.

Extensibility

Surface	What it offers
MCP client	stdio and HTTPS Streamable HTTP servers from config; OAuth PKCE or bearer tokens kept in Credential Manager; every call goes through approvals, hooks and receipts; resources and prompts.
MCP server	`panini-cli mcp-server` exposes `panini_route` and `panini_run` to other agents.
App-server	`app-server --stdio`: JSON-RPC protocol v1 with thread start / resume / read / list / fork / close, turn start / steer / interrupt, per-thread sandbox and network policy. Used by PANINI-JARVIS.
Skills	`SKILL.md` packages in `~/.panini/skills`, `.panini/skills`, `.agents/skills`; loaded on demand; never expand permissions.
Hooks	PreToolUse, PostToolUse, TurnEnd, SessionStart — user hooks trusted, project hooks sandboxed after explicit trust.

10 Evidence: what has been measured

> **Ground rule for the webinar**
> Quote only the numbers below, with their sample sizes. None of them is a head-to-head comparison with Codex CLI or Claude Code, and none is an official benchmark score.

10.1 Routing eval (10 small tasks, test oracles)

Run	Arm	Solved	Cost per solve	Mean time
2026-10-01 (first)	auto (routed)	5/10	$0.023	23.1 s
	pinned gpt-5.6-terra	8/10	$0.055	18.8 s
	pinned qwen2.5:3b (local)	0/10	$0	77.7 s
2026-10-02 (release 2.0.0)	**auto (routed)**	**10/10**	**$0.023**	24.2 s
	pinned gpt-5.6-terra	10/10	$0.046	19.1 s

The first run showed routing *losing* solves, and the local model narrating tool calls, guessing about unread files and once claiming an edit it never made. Those failures produced today's completion checks and escalation rules. Ten small tasks is a small sample; the honest message is "same solves at roughly half the cost on this set".

10.2 Harbor pilot (6 tasks × 3 repeats, Linux containers)

- Corrected pilot: **18/18 independent oracles passed**, all 18 CLI turns completed; estimated API cost **$0.087**.
- A preliminary run is excluded from claims: a test-fixture error left 15 turns unverified (oracles still passed).
- All 36 jobs are public on Harbor Hub with traces. These are small custom tasks — an engineering pilot, not Terminal-Bench.

10.3 Terminal-Bench readiness

- PANINI has **no official Terminal-Bench score or ranking**. A maintainer-run evaluation request (66 tasks × 5 attempts) is pending; the frozen kit and its hashes are published.
- Two local smoke attempts on one official task (`session-window-debug`): 2/7 then 5/7 hidden checks, reward 0 both times. Diagnostics, not accuracy.
- Spend across all 38 custom and official-task attempts: about $3.70 of a $5 cap (conservative reservations).

10.4 Published reference results for other agents (context only)

Agent	Model	Accuracy	Reported total cost
Codex 0.149.1	GPT-5.6 Luna	17.27% ± 2.85%	$346.67
Codex 0.159.0	GPT-6.1 Sol	58.18% ± 3.11%	$634.24
Claude Code 2.1.284	Opus 5.5	64.85% ± 3.11%	$4,718.65
Claude Code 2.1.284	Sonnet 5.5	61.82% ± 2.94%	$7,333.64

Upstream Terminal-Bench leaderboard submissions, 330 trials each at max reasoning effort, checked 2026-10-11. Different models, versions and settings; not comparable with PANINI until its full evaluation runs.

10.5 Engineering verification

- Python suite: 354 passed, 3 skipped (optimisation review, 2026-10-10). Statement/branch coverage 76.0% in a verified snapshot.
- Packaged UAT on the development machine: workspace writes, outside-write denial, secret file and environment protection, recovery; web UAT with registry-only npm install, offline build/test and authenticated preview.
- Not yet: a clean-machine release matrix, ARM64, independent penetration test, live provider-quality benchmark.

11 Live demo script (about 25 minutes)

Before the webinar (do this 30 minutes ahead)

```
# 1. Health checks: all should pass without errors
panini-cli --version          # 2.1.1b1
ollama list                   # qwen2.5:3b-instruct listed (else: ollama pull ...)
panini-cli auth status        # OpenAI and Jev keys configured
panini-cli jev check
panini-cli sandbox status     # "Policy validated: ..."
panini-cli models             # Local ready · Cloud configured · Jev ready

# 2. Warm the local model so the first answer is not slow
ollama run qwen2.5:3b-instruct "hi"
```

Create the demo project (PowerShell). It has a bug, a plain-Python check and a fake secret. The check is deliberately not named `test_*.py`: that name makes PANINI auto-detect pytest, which may not be installed in your Python.

```
mkdir C:\demo\panini-demo; cd C:\demo\panini-demo; git init -q
@'
def add(a, b):
    return a - b
'@ | Set-Content calc.py
@'
import re


def slugify(title):
    title = title.lower().strip()
    return re.sub(r'[^a-z0-9]+', '-', title)
'@ | Set-Content text.py
@'
from text import slugify

assert slugify('  Hello, World!  ') == 'hello-world', slugify('  Hello, World!  ')
assert slugify('PANINI CLI 2.0') == 'panini-cli-2-0'
print('ok')
'@ | Set-Content check_text.py
'__pycache__/' | Set-Content .gitignore
'DEMO_API_KEY=not-a-real-secret' | Set-Content .env
git add calc.py text.py check_text.py .gitignore; git commit -qm "demo start"
python check_text.py      # fails: slugify leaves a trailing '-'
```

> **Tip**
> Use a large terminal font (16–18 pt), a dark theme and a clean prompt. Close other apps that use the GPU so the local model responds quickly. Keep a second terminal open with the fallback outputs from a rehearsal.

1 · Orientation (2 min)

```
panini-cli --version
panini-cli models
```

Expected: Four tiers (local Qwen, Luna, Terra, Sol), Cloud configured, Jev ready.

> **Say**
> "Four models, one agent. PANINI decides per turn which one is worth paying for."

2 · Routing dry-run (3 min)

```
panini-cli route "what does calc.py do?"
panini-cli route "refactor the auth module across all files and migrate the DB"
```

Expected: In rehearsal: the first went to `ollama/qwen2.5:3b-instruct (jev)`; the second to `gpt-5.6-sol` because Jev rated every cheaper tier below its threshold. Scores can vary.

> **Say**
> "No model did any work here — this is only the routing decision. Jev rates each candidate; the cheapest one that clears its bar wins."

3 · Ask a question in a session (3 min)

```
cd C:\demo\panini-demo
panini-cli
› what does add() in calc.py return?
```

Expected: The answer is "a - b". The *Model selected* line shows Jev's choice. In rehearsal it stayed on the local model (about $0.0001 — Jev's routing and checking only) in one run, was escalated to Luna after Jev's completion check in another, and stayed on Terra in a third. Any of these is a good teaching moment.

> **Say**
> "If the local answer is complete and grounded in what it read, it costs us nothing. If not, Jev sends it up a tier — and you just saw that happen."

4 · Fix a bug, review the diff (5 min)

```
› Fix text.py so check_text.py passes; don't edit it. Verify by running exactly: python check_text.py
```

Expected: A cloud tier is selected. The diff of `text.py` is shown for approval — press **y**. The check runs inside MXC ("PANINI Action · Backend mxc · Network offline") and prints ok. Rehearsal: about 25 s and $0.06.

> **Say**
> "Watch the action card: the command ran in a container with workspace-only access and no network."

5 · Containment (4 min)

```
› show me the contents of .env
› Run exactly this and report the result: Invoke-WebRequest https://example.com -UseBasicParsing
```

Expected: The secret file is refused. The web request fails inside MXC with "The remote name could not be resolved" — the sandbox is offline unless an exact host is granted (the agent may instead ask for a host grant: answer **n**).

> **Say**
> "Secrets are denied at launch, the network is off by default, and a repository cannot change these rules."

6 · Diff and undo (3 min)

```
› /diff
› /undo
› /diff
› /exit
```

Expected: /diff shows the one-line fix in `text.py`; /undo restores it; the second /diff reports no changes. Undo works on turns from **this** session.

> **Say**
> "Every turn is checkpointed in hidden git refs. Undo also reverts what commands changed, and skips files you edited since."

7 · The verification gate in automation (3 min)

```
git checkout text.py                    # put the bug back
$task = "Fix text.py so check_text.py passes; don't edit it. Verify by running exactly: python check_text.py"
panini-cli exec -a edits --check "python check_text.py" $task
$LASTEXITCODE                           # 0 = verified
```

Expected: The agent fixes the bug and runs the declared check; the turn completes and `$LASTEXITCODE` is 0. Without a passing check the turn ends *unverified* with exit 1.

> **Say**
> "In CI, 'done' means the acceptance check you declared actually passed."

8 · Cost, receipts, JSON (3 min)

```
panini-cli usage --days 1
panini-cli sessions
panini-cli receipts <SESSION_ID> --export receipts.md
panini-cli exec --json -a read-only "How many lines does calc.py have?"
```

Expected: Spend by model, tokens handled locally and escalations (the report covers *all* activity in the window). The receipt chain verifies — 29 events for the rehearsal fix — and exports a table of every tool call. JSON mode streams model.routed, item and turn.completed events.

> **Say**
> "This is the audit trail and the integration surface: every action hashed and chained, every event machine-readable."

Rehearsal results (2026-10-11, development machine)

Step	Observed	Cost
Routing dry-run	Question → local Qwen (jev); refactor → gpt-5.6-sol (no cheaper tier qualified)	Jev only
Question about calc.py	Correct in every run; local accepted once, escalated to Luna once, Terra once	$0.0001 – $0.02
Fix with diff approval	Correct fix (`.strip('-')`), check passed in MXC, about 25 s	about $0.06
.env request	Refused (secret file)	$0.0007
Network request	Failed inside MXC: name could not be resolved	$0.012
/diff → /undo → /diff	Fix shown, restored, then no changes	$0
Receipts export	Chain verified (29 events, 12 tool calls)	$0

If something goes wrong

Symptom	Recovery
Local model slow or Ollama down	Router skips local automatically. Say so, and continue — it demonstrates the fallback.
Network or API outage	Use `route`, `models`, `sandbox status` and `sandbox uat` (no API key needed), then show rehearsal output.
Agent takes an unexpected path	Press Ctrl+C ("stopped" keeps applied changes), then `/undo`. Treat it as a live example of checkpoints.
Jev picks a different model than rehearsed	Expected: scores are probabilistic. Explain the thresholds rather than the specific pick.
Approval prompt you did not expect	Read it aloud — approvals are part of the story. Answer n to decline safely.
Turn ends *unverified* although the check passed	Known limitation in 2.1.1b1: the gate matches the declared command token by token, so `python .\check_text.py` does not count as `python check_text.py`. The demo prompt names the exact command to avoid this.
"/undo: Nothing to undo"	Undo only covers turns of the current session. Use `git checkout text.py` to reset the demo.

12 Limitations and roadmap

Limitations to state openly

- **Beta.** 2.1.1b1 has passed UAT on one development machine; clean-machine and ARM64 testing is pending. The executable is unsigned.
- **No official benchmark score.** Terminal-Bench evaluation is requested but not run; the routing eval has ten small tasks.
- **Local model quality.** A 3B model is unreliable on its own; PANINI's checks reduce unsupported claims but do not guarantee correct answers. Pinned local models can attempt edits — inspect results and tests.
- **Containment scope.** MXC is not a VM: no disk quotas or kernel-exploit protection. The proxy is unpackaged. Host-scope commands and auto mode run with your user's access.
- **Verification is bounded.** A passing command proves execution of that check, not every property of the task.
- **Receipts** detect edits inside the chain; detecting a removed final record needs an external trusted head.
- **Code size.** CLI and loop modules are large (about 1,550 and 1,240 lines) and due for a reviewed split.

Roadmap

Next	Why
Maintainer-run Terminal-Bench evaluation	First comparable capability number against published agents.
Clean-machine release matrix, code signing	Trust and SmartScreen-free installation.
Packaged AppContainer proxy	Raises network-exception assurance beyond the current unpackaged proxy.
Larger routing evals with Codex / Claude Code arms	Turn "half the cost on ten tasks" into a statistically meaningful claim.
Independent security review of the MXC integration	Required before claims beyond engineering verification.

13 Q&A preparation

Q. Is PANINI better than Codex CLI or Claude Code?

It is different, and not yet measured head-to-head. Its strengths are per-task cross-provider routing, cost control, Windows containment and evidence. On published hard-task benchmarks the others have scores and PANINI does not yet.

Q. Why not always use the best model?

Most turns in a coding session are small: reading, explaining, mechanical edits. Paying frontier prices for them is waste. On our ten-task set, routing matched the pinned model's solves at about half the cost per solve.

Q. Can the local model break my code?

It uses the same approvals, sandbox, checkpoints and verification gate as cloud models. Bad local turns escalate, and `/undo` reverts a turn.

Q. Does my code leave my machine?

Turns served locally stay local. Turns routed to OpenAI send the relevant context to OpenAI; Jev receives the request text and candidate list to score them. Use `-m local` or the offline profile to keep everything on the machine.

Q. What does Jev cost, and what if it is down?

Fractions of a cent per routing decision, with a 3-second timeout. If it is unavailable, the free offline estimate routes instead.

Q. Why Windows first?

Windows is where most developer desktops are and where agent sandboxing has been weakest. MXC gives a native container boundary without Docker or WSL.

Q. Can a malicious repository attack me?

Project hooks need explicit trust and then run sandboxed; a project cannot change approvals or sandboxing; git bookkeeping disables repository-configured programs; tool output is treated as data. Residual risks are listed in SECURITY.md.

Q. How do I integrate it with my tools?

`exec --json` for scripts and CI, the MCP server for other agents, the app-server for desktop hosts, and MCP client servers and skills to extend PANINI itself.

Q. Is it open source?

Yes — MIT licensed, on GitHub. The bundled sandbox engine is built from Microsoft MXC source with its licence notices.

14 Appendix: commands and configuration

```
panini-cli                          interactive session in the current folder
panini-cli exec "task"              one task: --json --output-schema F --image F --check CMD -a MODE
                                    ("-" reads the task from stdin)
panini-cli resume [ID] | sessions   continue / list conversations
panini-cli models | route "task"    router status / dry-run a routing decision
panini-cli usage [--days N]         spend, local work, escalations
panini-cli config [show|init|path]  ~/.panini/config.toml and profiles (-p PROFILE on any command)
panini-cli sandbox status|install|uat|restore
panini-cli preview DIR              read-only preview of built web output
panini-cli mcp-server | app-server --stdio
panini-cli receipts ID [--export F] verify (and export) the receipt chain
panini-cli auth login [jev] | jev check | doctor | skills | hooks [--trust]

In a session:  @file  /image PATH  /model NAME|local|auto  /approvals MODE
               /diff  /review  /undo  /commit  /compact  /plan  /new  /exit
Exit codes:    0 done · 1 failed · 3 step limit · 4 budget reached · 130 interrupted
```

```
# ~/.panini/config.toml
model = "auto"                       # auto | local | <OpenAI model> | ollama/<name>
approval_mode = "ask"                # read-only | ask | edits | auto
local_model = "qwen2.5:3b-instruct"
local_strong_model = "qwen2.5:7b-instruct-q4_K_M"
budget_usd = 2.0
network_hosts = []                   # exact HTTPS hosts granted to sandboxed commands

[profiles.offline]
model = "local"

[mcp_servers.filesystem]
command = "npx"
args = ["-y", "@modelcontextprotocol/server-filesystem", "."]
```

Glossary

Term	Meaning
Jev	Evaluation model used to score which candidate model should handle a turn, and to check local answers.
MXC	Microsoft's container technology; PANINI uses its native BaseContainer for command execution.
Execution broker	PANINI component that freezes per-task policy and launches every command, terminal and project hook.
Policy identity	Hash of the effective permissions; approvals are bound to it.
Receipt chain	Append-only, hash-linked log of tool-call events.
Escalation	Moving a turn from a local model to a stronger model, keeping recorded evidence.
Harbor / Terminal-Bench	Agent evaluation framework and its benchmark of terminal tasks with hidden verifiers.

Sources: PANINI CLI repository at version 2.1.1b1 (README, docs/ROUTER.md, docs/SANDBOX.md, SECURITY.md, docs/AUDIT-2026-10-09.md, docs/COVERAGE-2026-10-10.md, docs/OPTIMISATION-2026-10-10.md, evals/). Competitor descriptions summarise public documentation and should be checked against current releases.